

Experiment – 1.3.2

Student Name: Yatharth Bhaskar

UID: 24IBD70014

Branch: AIT-CSE (Data Science)

Section/Group: 24BDS-4 NTPP (A)

Semester: 5th

Date of Performance: 22 July 2026

Subject: Full Stack-II

Subject Code: 24CSP-337

1. Aim:

To implement role-based access control and secure application routes based on user permissions.

2. Software Required:

- React.js
- React Router
- State management (Context API / Redux)

3. Objective:

- To understand authorization mechanisms in applications
- To implement RBAC for access control
- To protect routes based on user roles
- To dynamically render UI elements based on permissions

4. Theory:

While authentication verifies user identity, **authorization** determines what actions a user is permitted to perform. Role-Based Access Control (RBAC) is a widely used model where users are assigned roles, and each role has defined permissions.

Common roles include:

- Admin
- Editor
- Viewer

RBAC ensures:

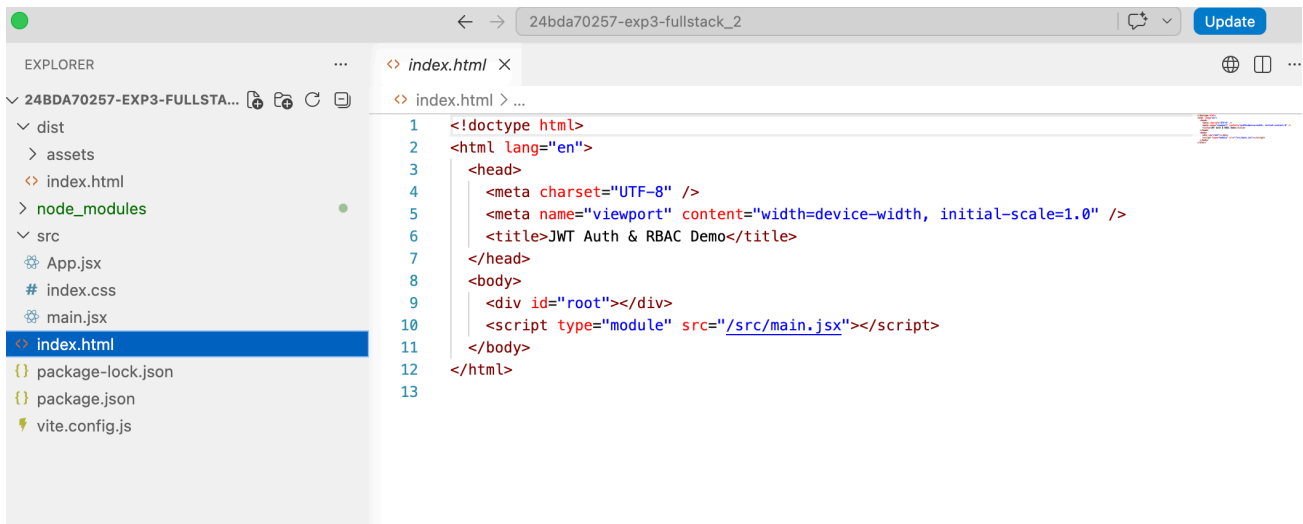
- Controlled access to sensitive features
- Simplified permission management
- Improved system security

Route protection is implemented to restrict navigation to certain pages unless the user has appropriate permissions.

5. Procedure/Algorithm:

- Define user roles (Admin, Editor, Viewer)
- Store role in authentication state
- Implement protected routes using React Router
- Restrict access based on role
- Conditionally render UI elements (buttons, pages)
- Redirect unauthorized users

6. Code:



```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>JWT Auth & RBAC Demo</title>
7   </head>
8   <body>
9     <div id="root"></div>
10    <script type="module" src="/src/main.jsx"></script>
11  </body>
12 </html>
13
```

7. Output:

JWT Auth + RBAC

Login

Secure Login

This demo uses a mock JWT stored in localStorage to simulate stateless authentication.

Username

Password

Login

Invalid credentials. Try admin/admin123, editor/editor123, or viewer/viewer123.

8. Conclusion:

- Role-based access control implemented
- Secure navigation across application
- UI dynamically adapts based on user role

9. Learning Outcomes:

- Understand the difference between authentication and authorization.
- Learn the concept of Role-Based Access Control (RBAC).
- Implement role-based permissions in web applications.
- Protect routes based on user roles and access rights.
- Enhance application security through controlled user access.